Roll Call
Documentación de Análisis

Miguel Gómez Casado
(a.k.a. Crazy Jäger)

[logos]

Resumen

Este documento surge como análisis de un proyecto software independiente con el objetivo secundario de proporcionar una muestra física y palpable de mi forma de trabajar. La idea del proyecto en sí surge como resultado de la necesidad de crear una aplicación Android que permita pasar lista a un grupo de personas pertenecientes a un grupo para así poder llevar un control de asistencia de una forma cómoda y sencilla.

Este documento busca proporcionar una documentación formal a un proyecto sencillo y pequeño que defina claramente el alcance funcional del proyecto, así como su estructura y forma de abordar el proyecto. Con ello en mente, tras una breve introducción, se procederá a presentar los requisitos del proyecto, los modelos del aplicativo resultado del análisis, los casos de uso, y la arquitectura y diseño finales del sistema.

Por último, hay que mencionar que este proyecto sigue una estrategia de desarrollo incremental, por lo que los contenidos de este documento pueden verse modificados según surjan nuevas especificaciones y modificaciones a realizar, sobre todo en fases iniciales al proyecto, que afectaras especialmente a los últimos apartados del documento.

Contenido

Resumen 1

1. Trasfondo y solución propuesta 3

Glosario ..... **¡Error! Marcador no definido.**

1. Trasfondo y solución propuesta

Durante mucho tiempo, he practicado judo. Al llegar al cinturón negro, se abrió un nuevo camino donde, además de seguir aprendiendo como alumno, podía impartir clases como profesor a otros que buscasen adentrarse en este deporte. Al principio, los grupos a los que impartía clases eran muy reducidos, con no más de 10 personas, pero a medida que ganaba experiencia y se me abría la posibilidad de impartir clases en pabellones más grandes, el número de alumnos también incrementaba. Este año, junto con otro compañero, empezamos a impartir clases a un grupo de tamaño considerable. Hasta ahora, una hoja de papel o una tabla en una hoja de cálculo era suficiente para llevar el control de asistencia del alumnado, pero, ante un grupo de estas dimensiones, la utilización de una hoja de cálculo se volvía bastante más aparatosa. A este problema se le suma el hecho de la necesidad de reducir objetos materiales tangibles, como el papel y el bolígrafo, debido a la Covid-19, motivo por el cual llevar un estricto control de asistencia es también una prioridad en toda actividad que tenga un número considerable de participantes, especialmente en un deporte de contacto. A raíz de todo esto, aparece la idea de crear una aplicación móvil que facilite pasar lista a los asistentes.

Inicialmente, este era solo un pequeño proyecto personal que tenía como objetivo facilitar el desarrollo de una actividad que realizaba con cierta regularidad. A consecuencia de ello, iba a tratarse de un pequeño desarrollo con baja calidad y sin ningún esfuerzo añadido en hacer una documentación formal como ésta. A medida que pasaba el tiempo, sin embargo, se hizo evidente que este proyecto abría la posibilidad de hacer pública mi forma de operar en un aspecto de la ingeniería de software que muchos compañeros de la profesión informática encuentran desagradable e incluso tedioso; pero que, como en cualquier proceso de ingeniería, puede llegar a ser de vital importancia para un proyecto, especialmente si dicho proyecto tiene un tamaño considerable o va a pasar por muchas manos durante su desarrollo y mantenimiento. La percatación de este hecho sería lo que marcaría el punto de inicio del desarrollo de esta documentación y, posteriormente, el de la aplicación.

El objetivo motivaba la realización del proyecto, y la posibilidad de mostrar mis habilidades en este campo motivaban la realización de la documentación de éste, así como su publicación. Con la decisión tomada de hacer el proyecto público, se abría una última puerta que había querido explorar desde hace ya algún tiempo: los derechos de autor y las licencias.

Conozco, gracias a amigos y familiares, historias de informáticos que, con una motivación e ilusión como la mía, habían desarrollado por cuenta propia un software que les era después arrebatado, a menudo por el propio cliente, quien se atribuía a sí mismo la autoría del trabajo y lo explotaba sin ningún respeto hacia quienes habían invertido tiempo y esfuerzo en realizarlo. Por eso, decidí aprovechar esta ocasión para empezar a conocer más acerca de este mundo legal que no me atrae demasiado pero que, por los motivos ya descritos, es interesante conocer. La decisión de qué licencia utilizar fue bastante sencilla de tomar. Buscaba empezar a adentrarme en el mundo de las licencias sin tener que pagar demasiado, ya que lo que realmente buscaba era

asegurar de una manera más formal mi autoría sobre el proyecto, sin importarme demasiado quién decidía finalmente hacer uso de él. Con esto, me incliné directamente a la Licencia Pública General, comúnmente referida por sus siglas en inglés GPL (*General Public Licence*) del proyecto GNU. Esta licencia es suficiente para atribuirme la autoría de este proyecto, tal y como yo quería. También resulta beneficioso para cualquier persona que buscase una aplicación con estas características que no quisiera gastar dinero; un principiante en Kotlin, programación Android o, en general, en el desarrollo de software que buscase un ejemplo práctico de una aplicación con estas características; y, finalmente, para una persona que pudiese estar interesado en comprobar cuál es mi metodología en el desarrollo de software. Todo esto es posible gracias al hecho de que la licencia GPL de GPU, además de concederme los derechos de autor del código, autoriza expresamente la explotación de la aplicación y la posibilidad de visualizar, modificar y distribuir el software con total libertad (siempre respetando la autoría, como se ha descrito anteriormente).

Pasando a hablar de la solución software propuesta para el problema descrito al inicio de este apartado, el aplicativo se tratará de una aplicación móvil para el sistema operativo Android desarrollada en el lenguaje de programación Kotlin que permitirá pasar lista a uno o más grupos de personas. Además de esta funcionalidad principal, la aplicación almacenará información adicional de las personas del grupo, como datos de contacto; y permitirá la exportación de los datos almacenados a un formato fácilmente visible en una aplicación de hoja de cálculo, permitiendo así al usuario elaborar un informe más detallado de asistencia si así lo requiriesen.

La metodología de desarrollo a seguir en este proyecto será un desarrollo incremental. El desarrollo incremental se caracteriza por priorizar la compleción un primer objetivo funcional, pasar a producción, e ir completando más adelante los objetivos funcionales restantes. Esto permite que, desde el momento que se despliega una primera versión de la aplicación, ésta ya cuenta con una parte funcional, permitiendo detectar errores o posibles desviaciones de los objetivos iniciales antes de que la toma de acciones pertinentes para subsanar estos errores sea demasiado costosa o complicada.

Esta decisión se tomó debido a los siguientes aspectos:

- **El proyecto no requiere coordinación.** Este proyecto es un desarrollo propio realizado por una única persona. Eso significa que la idea siempre es la misma y proviene de la misma persona, permitiendo así tirar hacia un tipo de desarrollo más tradicional frente a un desarrollo más en línea con los desarrollos ágiles y la toma de decisiones en equipo, como SCRUM.
- **Interés en tener una documentación formal.** En el desarrollo ágil también existe una documentación, pero es mucho más resumida e informal. Esto es porque parte de la filosofía ágil es la dar más importancia a la autodocumentación frente a la idea más tradicionalista de tener un documento como el presente que contenga toda la información formal del proyecto. Como, en nuestro caso, interesa destacar este documento, una vez más, interesa una metodología algo más tradicional.

- **Tradicional, pero flexible.** Aunque este tipo de desarrollo puede considerarse algo tradicional, ya se encuentra a medio camino entre la idea proveniente de la ingeniería industrial acerca de como afrontar un desarrollo software -también conocido como modelo en cascada- y las metodologías ágiles, que surgen de la necesidad de añadir un enorme grado de flexibilidad al desarrollo software, dadas sus características de cambio y evolución constante propias del mundo en el que vivimos.
- **La aplicación busca aportar utilidad.** Como ya se mencionó al inicio de esta sección, el objetivo de este proyecto es, desde sus orígenes, hacer frente a una necesidad real. Por este motivo, es interesante contar con una aplicación funcional, aunque dicha funcionalidad sea parcial, para ir solucionando este problema poco a poco, ya que sus características lo hacen posible e incluso conveniente.

Resumiendo, este proyecto y su documentación surgen a raíz de la necesidad de facilitar el control de asistencia en un grupo de alumnos de judo. A esta necesidad se le suma la posibilidad de hacer este proyecto público, para así potenciar mi currículum, mostrando mis habilidades y forma de trabajar. A mayores, añadimos la posibilidad de tener un primer contacto con las licencias y los derechos de autor. La metodología a utilizar en el proyecto será incremental debido a que es un desarrollo pequeño y realizado por una sola persona, existe un interés en hacer una documentación formal, alejándolo de metodologías más ágiles. También aporta una flexibilidad que no tiene el modelo tradicional o en cascada, que es interesante para absorber los peligros que supone la incertidumbre de eventos que pueden aparecer durante el proyecto. Finalmente, se busca contar con una versión funcional, aunque dicha funcionalidad sea parcial, en cuanto antes, para poder aprovechar el beneficio que la aplicación pretende aportar.

Todo esto dicho, pasamos a la siguiente sección, donde realizaremos un análisis de los requisitos del proyecto.

2. Análisis de requisitos

Al inicio de todo proyecto software, es importante conocer los requisitos del mismo, ya que condicionarán tanto la forma en la que se irá desarrollando, como el resultado final. En este apartado, describiremos los requisitos del sistema, así como sus objetivos funcionales. Para ser más concretos, en los siguientes subapartados veremos los objetivos funcionales, los requisitos funcionales, los requisitos no-funcionales y, finalmente, los requisitos de información.

Previo a empezar a describir los requisitos, explicaremos un poco la notación utilizada en este apartado. En primer lugar, es importante explicar que los requisitos suelen estar relacionados con los objetivos funcionales. Por este motivo, para facilitar en la identificación de los objetivos que son referenciados, los objetivos se referenciaran mediante etiquetas de la forma [OBJ-XX], donde "XX" es el número del objetivo. Los requisitos, por lo tanto, serán de la notación [R-XXYY], donde "XX" coincidirá con el objetivo funcional referenciado e "YY" será el número del requisito de ese mismo tipo asociado al objetivo.

Un ejemplo de esto sería: [RF-105], que sería el quinto requisito funcional (RF) del objetivo 1 ([OBJ-01]). Como se ha podido observar en el ejemplo, si el primer número del objetivo es "0", se omitirá en las etiquetas de los requisitos para mejorar la legibilidad.

2.1. Objetivos Funcionales

Este subapartado busca describir brevemente los objetivos funcionales del sistema. Los objetivos funcionales describen una serie de hitos o, como su nombre indica, objetivos generales que se espera conseguir con la aplicación. Esto nos permite definir una primera separación o modularización del proyecto, estableciendo una lista de requisitos segmentada, así aumentando su grado de legibilidad al añadir un mayor grado de organización.

Los objetivos funcionales son, habitualmente, un número reducido de apartados a partir de los cuales surgen los requisitos funcionales, requisitos no funcionales y requisitos de información, que aumentan la granularidad del objetivo funcional al añadirle un mayor grado de detalle.

Objetivos funcionales	
OBJ-01	Gestión de grupos e integrantes
OBJ-02	Realizar un control de asistencia de los integrantes de un grupo
OBJ-03	Exportar datos de un grupo

2.2. Requisitos funcionales

Este subapartado describe brevemente los requisitos funcionales de la aplicación. Los requisitos funcionales hacen referencia a una acción concreta que se espera que la aplicación pueda realizar. Como se describe en el glosario, en el análisis de requisitos suele emplearse la palabra sistema o el sistema para referirse a la aplicación, ya que en este punto nos encontramos en un nivel de abstracción independiente de cómo se va a llevar a cabo el proyecto, aunque ya hayamos decidido como hacerlo. Por este motivo, los requisitos funcionales suelen presentarse como una oración que tiene a "el sistema" como sujeto. Los requisitos funcionales describen las acciones o funciones que el sistema debe tener para cumplir los objetivos funcionales. En definitiva, describen el "qué hace el sistema".

A partir de los requisitos funcionales se desarrollarán en los apartados siguientes los casos de uso del sistema, ya que describen la funcionalidad concreta que debe ser proporcionada.

Requisitos Funcionales	
RF-101	El sistema permitirá crear un nuevo grupo
RF-102	El sistema permitirá añadir un nuevo integrante en un grupo
RF-103	El sistema permitirá modificar los datos de un integrante
RF-104	El sistema permitirá eliminar un integrante de un grupo
RF-105	El sistema permitirá eliminar un grupo
RF-106	El sistema permitirá visualizar los datos de un grupo
RF-107	El sistema permitirá visualizar los datos de un integrante
RF-201	El sistema permitirá realizar un control de asistencia sobre un grupo
RF-202	El sistema permitirá modificar un control de asistencia ya realizado
RF-301	El sistema permitirá exportar los datos de asistencia de un grupo

2.3. Requisitos no-funcionales

Este subapartado describe brevemente los requisitos no-funcionales de la aplicación. Los requisitos no funcionales establecen una serie de restricciones acerca de la forma en la cual se debe alcanzar el objetivo funcional con el que se asocian. Esto permite incluir como requisitos detalles que se escapan al ámbito de los requisitos funcionales, como pueden ser tecnologías, herramientas o metodologías concretas que se deben emplear en la realización de una acción necesaria para cumplir el objetivo, describiendo el "cómo lo hace el sistema".

Estos requisitos pueden modificar ligeramente la forma en la cual se realizan los casos de uso, pero su impacto es bastante pequeño en esta parte del proyecto. Su importancia se ve incrementada, sin embargo, en las últimas fases del análisis del proyecto, así como el desarrollo de la propia aplicación resultante.

Existen algunos requisitos funcionales, como el lenguaje de programación o un requisito proveniente del entorno en el que operará el sistema que afecta a la totalidad del sistema. Estos requisitos son importantes y deben quedar reflejados, a pesar de no relacionarse con ningún objetivo funcional. Estos requisitos no-funcionales en particular tendrán el número "0" como prefijo a su identificador.

Requisitos no-funcionales	
RNF-001	El sistema se desarrollará en el lenguaje de programación Kotlin
RNF-002	El sistema será compatible con la versión 8 del sistema operativo Android
RNF-101	Un grupo puede ser importado a la aplicación para crearlo
RNF-102	Un grupo podrá ser creado manualmente integrante a integrante
RNF-103	Un grupo puede ser importado en forma de texto que seguirá el formato indicado en el fichero *example.new_group.txt*
RNF-104	Un grupo puede ser importado desde un fichero CSV que seguirá el formato indicado en el fichero *example.new_group.csv* o *example.group.csv*
RNF-105	Un grupo puede ser importado desde un fichero XML que seguirá el formato indicado en el fichero *example.new_group.xml* o example.*group.xml*
RNF-106	Un fichero XML que represente un grupo seguirá la definición establecida en *crazyjaeger_rollcall_group.dtd*
RNF-107	Un grupo puede ser importado desde un fichero JSON que seguirá el formato indicado en el fichero *example.new_group.json* o *example.group.json*
RNF-103	La eliminación de un integrante no eliminará la información que se haya recogido del mismo
RNF-301	Los datos de un grupo se podrán exportar a un único fichero
RNF-302	Los datos de un grupo se podrán exportar a un fichero CSV que seguirá el formato indicado en el fichero *example.group.csv*
RNF-303	Los datos de un grupo se podrán exportar a un fichero XML que seguirá el formato indicado en el fichero *example.group.xml*
RNF-304	Un fichero XML que represente un grupo seguirá la definición establecida en *crazyjaeger_rollcall_group.dtd*
RNF-305	Los datos de un grupo se podrán exportar a un fichero JSON que seguirá el formato indicado en el fichero *example.group.json*

Los ficheros referentes a ejemplos referenciados en los requisitos no-funcionales podrán encontrarse en el directorio *example.data* que forma parte del proyecto y el archivo *crazyjaeger_rollcall_group.dtd* se encontrará presente en el directorio *doc* del proyecto. No obstante, para mayor comodidad, se incluirá en el anexo de este documento una copia del contenido de los ficheros referenciados.

A continuación, nos encontraríamos un apartado relativo a las llamadas Reglas de Negocio. Al igual que los requisitos no-funcionales, las reglas de negocio, provenientes del inglés *Buisness Rules*, establecen una serie de restricciones acerca de cómo debe realizar el sistema una acción concreta. La diferencia es que, al contrario de los

requisitos no-funcionales que establecen restricciones acerca de tecnologías o metodologías, las reglas de negocio establecen restricciones propias de la metodología utilizada en la organización que va a utilizar la herramienta. Si esta aplicación contara con diferentes roles de usuario, como una persona encargada de dar de alta a nuevos grupos y otra diferente encargada de llevar el control de asistencia, el requisito que marca la restricción de acciones realizables por cada rol iría en este apartado.

En el caso de esta aplicación, no necesitamos establecer una jerarquía de permisos y restricciones ni debemos ajustarnos a la forma de operar de una organización concreta. Debido a ello, no existirá esta sección de reglas de negocio, aunque si lo menciono para destacar que es bastante común encontrarlo en otros proyectos.

2.4. Requisitos de información

En este apartado describimos los requisitos de información. Los requisitos de información, en ocasiones también referidas como requisitos funcionales de información, establecen qué datos de los diferentes elementos presentes en el sistema se deben mantener durante el funcionamiento del mismo. Estos datos, adicionalmente, suelen coincidir con los datos que se deberán almacenar en memoria persistente posterior a su uso por el sistema. Mucho de lo que se incluye en esta serie de requisitos será lo que posteriormente se verá reflejado de una forma más gráfica en los diagramas de modelo de dominio, diagramas de entidad-relación o similares.

Los requisitos de información se suelen representar como un listado de la información que se debe manejar de cada uno de los elementos.

Requisitos de información	
RI-101	De cada grupo, el sistema almacenará: • El nombre del grupo • La descripción del grupo • Los integrantes que pertenecen al grupo
RI-102	De cada integrante, el sistema almacenará: • El nombre • Los apellidos • Fecha de nacimiento • Teléfonos de contacto
RI-201	De cada evento, el sistema almacenará: • El grupo sobre el que se aplica • La fecha y hora de realización del control • Una lista de las asistencias recogidas
RI-202	De cada asistencia, el sistema almacenará: • El usuario cuya asistencia ha sido controlada • Si el usuario estaba presente o no

3. Modelo de dominio

El modelo de dominio es un diagrama de clases que muestra de manera gráfica los elementos que forman parte del dominio del sistema. Estas clases surgen como resultado del análisis y podrán ser diferentes a las entidades reales que existirán en el sistema ya que el modelo de dominio permite abstraerse de las particularidades de la tecnología que se utilizará finalmente para desarrollar el sistema software. Además del diagrama, se incluirá un subapartado con las restricciones OCL (*Object Contraint Language*) aplicados al dominio.

[TODO]

3.1. Restricciones OCL

Las restricciones OCL establecen una serie de restricciones sobre los datos del dominio. Estas restricciones pueden ser invariantes, o pueden representar precondiciones y postcondiciones de la ejecución de una operación determinada.

[TODO]

4. Persistencia

[TODO]

5. Casos de uso

[TODO]

5.1. Diagrama de casos de uso

[TODO]

5.2. Casos de uso detallados

[TODO]

5.3. Diagramas de secuencia

[TODO]

6. Arquitectura

[TODO]

7. Diseño

[TODO]

8. Despliegue

[TODO]

Anexo

A. Glosario

- **El/este proyecto:** el desarrollo de este proyecto software en particular, incluyendo tanto la documentación formal presente en este documento, el manual de usuario que se incluirá al final del desarrollo y el propio aplicativo software que surgirá como producto final del proceso de desarrollo.
- **La aplicación:** el aplicativo para sistema Android que surgirá como producto final de este desarrollo. En palabras menos formales, el programa ejecutable y los componentes que lo forman.
- **El aplicativo:** mismo significado que la aplicación, descrito encima de este término.
- **El sistema:** mismo significado que la aplicación, descrito dos términos más arriba.
- **El usuario:** persona que utiliza la aplicación.
- **Modelo mental:** conjunto de ideas que uno tiene acerca de las entidades que le rodean. Cuando una persona oye la palabra "libro" en una conversación, a su mente acudirá una imagen de lo que entiende por "libro", permitiéndole entender el significado de la palabra. Esta imagen mental es una de millones que conforman lo que llamamos "modelo mental".
- **El Dominio:** es un conjunto abstracto de entidades que forman parte del sistema que tienen su equivalente en el modelo mental. Volviendo al ejemplo del libro utilizado para definir el modelo mental, la entidad que representa un libro tendrá, al igual que tiene un representante en el modelo mental, un representante en el dominio de una aplicación como puede ser el catálogo de una biblioteca. El "libro" de la aplicación de la biblioteca es diferente al "libro" del modelo mental de la persona que usa la aplicación, pero ambas entidades hacen referencia al mismo libro físico, a pesar de que todos podemos afirmar que el libro físico, la idea mental del libro y el libro del dominio son tres entidades diferentes.
- **Grupo:** componente del dominio que representa a un conjunto de integrantes. Su equivalente en el modelo mental sería el de un grupo de personas de los que es responsable el usuario y sobre el que desea llevar a cabo un control de asistencia.
- **Integrante:** componente del dominio que forma parte de un grupo. Su equivalente en el modelo mental sería el de una persona. El usuario está interesado en conocer si esta persona asistió o no a un evento organizado para el grupo al que pertenece.
- **Control de asistencia:** componente del dominio que representa un conjunto de asistencias. Su equivalente en el modelo mental sería el de el listado que contiene los nombres de los asistentes de un evento que el usuario utilizaría para anotar quien está "presente" y quien está "ausente"

- **Asistencia:** componente del dominio que representa a la asistencia de un integrante. Su equivalente en el modelo mental sería el de la anotación de "presente" o "ausente" en el momento de pasar lista a una persona concreta.

B. Contenido del fichero *crazyjaeger_rollcall_group.dtd*

TODO

C. Contenido del fichero *example.new_group.txt*

TODO

D. Contenido del fichero *example.new_group.cvs*

TODO

E. Contenido del fichero *example.new_group.xml*

TODO

F. Contenido del fichero *example.new_group.json*

TODO

G. Contenido del fichero *example.group.cvs*

TODO

H. Contenido del fichero *example.group.xml*

TODO

I. Contenido del fichero *example.group.json*

TODO